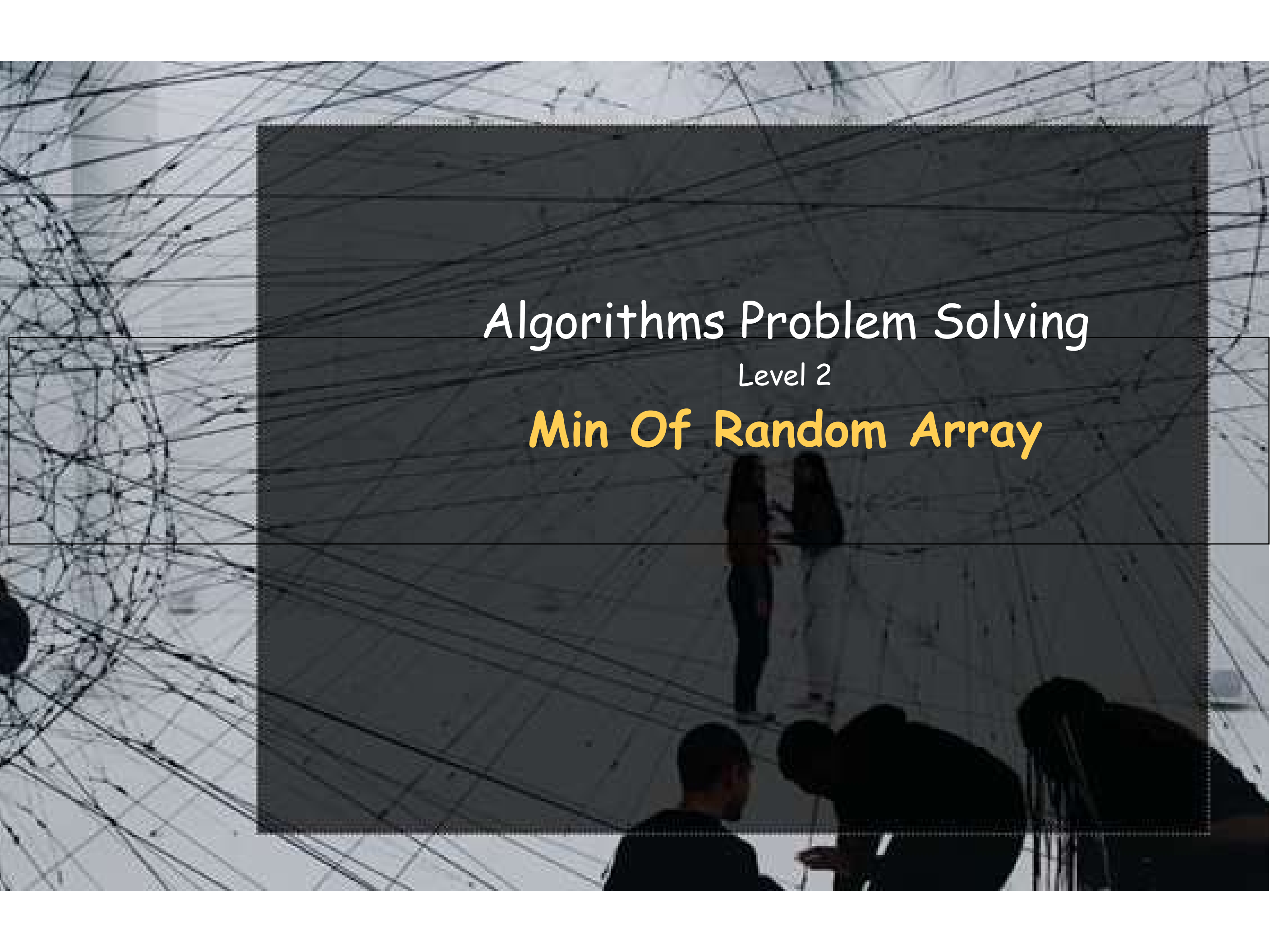


Algorithms Problem Solving

Level 2

Min Of Random Array

The background of the slide is a photograph of a modern building's interior. It features a large, curved glass ceiling that allows natural light to filter through, creating a pattern of light and shadow on the floor. In the lower part of the image, the silhouettes of several people are visible, some standing and some sitting, suggesting a collaborative workspace or a public area. The overall tone is professional and tech-oriented.

Algorithms Problem Solving

Level 2

Min Of Random Array

Definition

- ◆ A program that fills an array with random numbers from `1` to `100`, prints the array, and finds the minimum number stored in it.
- ◆ The program compares the array elements to determine the smallest value.

Explanation

- ◆ ``FillArrayWithRandomNumber()`` asks the user for the number of elements and fills the array with random numbers from ``1`` to ``100``.
- ◆ ``PrintArrayElements()`` prints all elements stored in the array.
- ◆ ``MinNumbInArray()`` searches for the smallest value in the array.
- ◆ The variable ``min`` is initialized with the first array element using ``arr[0]``.
- ◆ Each array element is compared with ``min``.
- ◆ If ``arr[i] < min``, the current element becomes the new minimum.
- ◆ After checking all elements, the function returns the smallest number.

Steps in main()

1. Create an array with a maximum size of `100`.
2. Ask the user for the number of elements.
3. Fill the array with random numbers from `1` to `100`.
4. Print all array elements.
5. Set `min` to the first array element `arr[0]`.
6. Loop through all array elements.
7. Compare each element with `min`.
8. If the current element is smaller, update `min`.
9. Continue until all elements have been checked.
10. Return and print the minimum number.

💡 Example

◆ Input:

Enter number of elements: 10

◆ Generated array:

42 68 35 1 70 25 79 59 63 65

◆ The program compares the numbers and finds that `1` is the smallest value.

➡ Result

◆ The program prints:

Array Elements: 42 68 35 1 70 25 79 59 63 65

Minimum Number is: 1

📌 Note

- ◆ ``min`` stores the smallest value found so far.
- ◆ The comparison:
 ``if (arr[i] < min)``
checks whether the current element is smaller than the current minimum.
- ◆ The program initializes ``min`` with ``arr[0]`` instead of ``0``.
- ◆ This is safer because the array could contain negative numbers in a different program.
- ◆ Starting with ``arr[0]`` ensures that the initial minimum is an actual element from the array.

⊘ Warning

- ◆ `FillArrayWithRandomNumber()` does not validate that `arrLength` is between `1` and `100`.
- ◆ If the user enters more than `100`, the program can access memory outside the array.
- ◆ `MinNumbInArray()` assumes that the array contains at least one element because it accesses `arr[0]`.
- ◆ The program does not call `srand((unsigned)time(NULL))`, so the random sequence may repeat between executions.
- ◆ The comment and example say `Max Number is: 79`, but this program is finding the minimum, so the correct result for the example is `1`.



Comparison

▨ `MaxNumbInArray()`

- ✓ Searches for the largest value.
- ✓ Uses ``arr[i] > max`` to update the maximum.
- ✓ Example: ``42 68 35 1 70`` → ``70``.

▤ `MinNumbInArray()`

- ✓ Searches for the smallest value.
- ✓ Uses ``arr[i] < min`` to update the minimum.
- ✓ Starts with ``arr[0]`` as the initial minimum.
- ✓ Example: ``42 68 35 1 70`` → ``1``.

✦ Summary

- ✓ The program fills an array with random numbers from `1` to `100`.
- ✓ `PrintArrayElements()` prints the array.
- ✓ `MinNumbInArray()` searches for the smallest value.
- ✓ `min` is initialized with `arr[0]`.
- ✓ Each element is compared with the current minimum.
- ✓ If a smaller value is found, `min` is updated.
- ✓ For the example array, the minimum number is `1`.

Thank you 😊